

UNIVERSIDAD EAFIT
ST0245 - Estructuras de Datos y Algoritmos

Informe Final

Practica I - Sorting Large Dataset

Ordenamiento de 100,000 palabras: QuickSort, HeapSort y Arbol AVL

Integrantes:
Johan Smith Tobon
Juan Camilo Estrada

Septiembre de 2026

Tabla de Contenido

Tabla de Contenido	2
1. Introduccion	3
2. Enfoque de implementacion	3
2.1 QuickSort	Error! Bookmark not defined.
2.2 HeapSort.....	Error! Bookmark not defined.
2.3 Arbol AVL.....	Error! Bookmark not defined.
3. Mediciones.....	3
3.1 Tiempo de ejecucion	4
3.2 Uso de memoria	4
4. Analisis comparativo.....	5
4.1 Complejidad teorica (Big O) vs tiempo real.....	Error! Bookmark not defined.
4.2 Por que el tiempo real difiere de la teoria	5
4.3 Seleccion final	5
5. Conclusiones	6
Anexos	6

1. Introduccion

El reto principal de este trabajo fue ordenar 100,000 palabras, y para lograrlo probamos tres caminos distintos: QuickSort, HeapSort y un árbol AVL. Un detalle clave es que creamos todas las estructuras y algoritmos desde cero en C++, sin recurrir a funciones integradas de las librerías estándar como `std::sort` o `std::priority_queue`.

Antes de empezar a medir tiempos, nos topamos con un problema técnico: el archivo que nos dieron estaba en formato UTF-16 (la codificación por defecto del Bloc de notas en Windows). Al intentar leerlo directamente en C++, la consola interpretaba los bytes extra como caracteres nulos intercalados entre cada letra. Tuvimos que hacer un paso previo de conversión a UTF-8 para limpiar el texto. Pues, Por suerte, los datos ya venían desordenados, así que no hizo falta barajarlos.

2. Enfoque de implementacion

2.1 QuickSort

Para este algoritmo elegimos como pivote el último elemento del subarreglo (`v[fin]`). La función `partition` recorre el vector usando dos punteros: uno (`j`) explora los datos y otro (`i`) marca el límite donde van quedando los valores menores al pivote. Cada vez que encontramos un elemento menor, avanzamos `i` y hacemos un intercambio (`swap`). Al terminar el recorrido, el pivote se ubica en su posición definitiva y el algoritmo se llama de forma recursiva para los subarreglos izquierdo y derecho.

2.2 HeapSort

A diferencia de otras estructuras basadas en nodos, el montículo (`heap`) vive directamente sobre el vector utilizando aritmética de índices: para cualquier posición `i`, su hijo izquierdo está en $2*i + 1$ y el derecho en $2*i + 2$.

La función `heapify` compara un nodo con sus hijos y, si uno de ellos es mayor, intercambia valores y baja por el árbol ajustando la estructura. Para construir el heap inicial recorremos el vector desde $n/2 - 1$ hacia atrás. Después, extraemos el elemento máximo repetidamente intercambiándolo con la última posición libre y reduciendo el tamaño del heap hasta que todo el vector queda ordenado.

2.3 Árbol AVL

En la implementación del AVL, cada nodo almacena su palabra, punteros a sus dos hijos y su altura (esta última para calcular el factor de balance en tiempo constante $O(1)$ sin tener que recorrer la estructura).

La inserción sigue la lógica estándar de un árbol binario de búsqueda, pero al regresar de la llamada recursiva se verifica el balance del nodo. Si se pierde el equilibrio, se aplica una de las cuatro rotaciones posibles (Simple Izquierda, Simple Derecha, Doble Izquierda-Derecha o Doble Derecha-Izquierda).

Un detalle importante durante las pruebas: Vimos que existía un bug en el caso Derecha-Derecha, donde la condición lógica estaba invertida y ejecutaba la rotación equivocada. Como estábamos insertando las palabras ya ordenadas para forzar el peor escenario, cada inserción caía exactamente en ese caso defectuoso. Esto hizo que el árbol perdiera su auto-balance y se degenerara en una lista enlazada de 100,000 niveles de profundidad, lo que ralentizó drásticamente el proceso sin lanzar errores de compilación.

Ya corregido el error, el árbol mantuvo su altura óptima y pues bastó con hacer un recorrido in-order para obtener las palabras ordenadas sin necesidad de pasos adicionales.

3. Mediciones

3.1 Tiempo de ejecución

Algoritmo	Complejidad teorica (Big O)	Tiempo real (promedio de 5 corridas)
QuickSort	$O(n \log n)$ promedio, $O(n^2)$ peor caso	187 ms (rango: 165-201 ms)
HeapSort	$O(n \log n)$ garantizado (sin peor caso)	297 ms (rango: 260-336 ms)
Arbol AVL (insercion + inorder)	$O(n \log n)$	130 ms (rango: 112-144 ms)

Las pruebas se corrieron en un equipo con procesador Intel Core i5-11400H de 11a generacion (6 nucleos, 2.70 GHz base), 24 GB de RAM, sistema operativo Windows, compilando con GCC 15.2.0 (MinGW-w64) desde CLion. Cada algoritmo se ejecuto 5 veces sobre el mismo dataset de 100,000 palabras; los valores de la tabla corresponden al promedio de esas 5 corridas, y se incluye el rango (minimo-maximo) para mostrar que tan estable fue cada medicion. El orden relativo se mantuvo consistente en todas las corridas: HeapSort siempre fue el mas lento y el AVL siempre el mas rapido.

3.2 Uso de memoria

Estructura	Formula teorica	sizeof por elemento	Memoria aprox. total
Vector (Quick/Heap)	$n \times \text{sizeof}(\text{std::string})$	32 bytes	3,200,000 bytes (~3.05 MB)
Arbol AVL	$n \times \text{sizeof}(\text{NodoAVL})$	56 bytes	5,600,000 bytes (~5.34 MB)

El nodo del AVL resulta más pesado porque, además de almacenar la palabra (los 32 bytes de `std::string`), necesita guardar la estructura del árbol: dos punteros (8 bytes cada uno) y la altura (8 bytes por alineación de memoria). Esto da 56 bytes por nodo (`sizeof(NodoAVL)`), donde los 24 bytes extra frente al vector son el costo puro de mantener las conexiones.

A esto se sumemosle la gestión de memoria:

- Vector: Reserva un solo bloque contiguo para las 100,000 palabras, optimizando el uso de la memoria caché de la CPU.
- AVL: Asigna cada nodo por separado con `new`, lo que dispersa la estructura por la RAM (fragmentación) y provoca saltos constantes de memoria al recorrerlo.

Así, la diferencia de rendimiento del AVL no es solo por el espacio que ocupa, sino por la pérdida de localidad de memoria.

4. Analisis comparativo

4.1 Complejidad teórica (Big O) vs. tiempo real

Aunque HeapSort garantiza $O(n \log n)$ en todo escenario y QuickSort puede degradarse a $O(n^2)$, en la práctica ocurrió lo opuesto: HeapSort tardó 297 ms y QuickSort 187 ms (un 50% más rápido). Al estar el dataset aleatorizado, QuickSort operó en su caso promedio ($O(n \log n)$). A igual complejidad asintótica, el Big O pierde precisión porque ignora las constantes ocultas, la cantidad real de instrucciones y la eficiencia con la que se accede a la RAM.

4.2 Por que el tiempo real difiere de la teoria

Localidad de memoria (Caché): QuickSort recorre el vector secuencialmente, permitiendo al procesador precargar datos en la caché. HeapSort, en cambio, salta entre índices distantes (i , $2i+1$, $2i+2$), provocando fallos de caché continuos.

Por qué ganó el AVL (130 ms): Aunque usa punteros dispersos, cada inserción solo recorre un camino muy corto desde la raíz hasta la hoja (≈ 17 niveles para 100,000 elementos) y las rotaciones son escasas. HeapSort realiza comparaciones y swaps repetidos en cada nivel durante sus 100,000 extracciones.

Costo de comparación: Comparar `std::string` no es $O(1)$ sino proporcional a la longitud de la palabra. Afecta a los tres por igual, pero demuestra que Big O mide operaciones abstractas, no el costo real de CPU.

4.3 Selección final

QuickSort (Elección recomendada): Es la mejor opción para ordenar una colección estática de una sola vez. Fue el más rápido entre los algoritmos de ordenamiento puro y consume solo 3.2 MB frente a los 5.6 MB del AVL.

Árbol AVL: Sería la opción ideal solo si el sistema fuera dinámico y requiriera búsquedas o inserciones continuas en $O(\log n)$ sin reordenar todo desde cero.

HeapSort: Garantiza $O(n \log n)$ en el peor caso y no usa memoria extra (ordena in-place), pero su acceso disperso a memoria ($2i+1$, $2i+2$) lo hizo el más lento de los tres en la práctica.

5. Conclusiones

Este trabajo deja una lección clara: la complejidad teórica (Big O) es necesaria pero insuficiente para predecir el rendimiento real. HeapSort, el único con $O(n \log n)$ garantizado en todo escenario, fue el más lento (297 ms) debido a que sus saltos de memoria ($2i+1$ / $2i+2$) provocan constantes fallos de caché frente al acceso secuencial de QuickSort (187 ms).

Asimismo, confirmamos que más memoria no implica menor velocidad: el AVL fue el más rápido (130 ms) pese a ser el más pesado (5.6 MB vs. 3.2 MB del vector), costo derivado de los punteros y la altura de cada nodo.

Lecciones clave del proyecto:

- Bugs silenciosos: El fallo en la rotación del AVL demostró que un error de lógica sintáctica (condición invertida) no destruye el programa, pero degrada el rendimiento drásticamente al convertir la estructura en una lista enlazada.
- Criterio de selección: No existe el "mejor" algoritmo en absoluto. QuickSort es la opción ideal para ordenar colecciones estáticas una sola vez con bajo consumo de memoria. En cambio, el AVL es superior si se requiere mantener una estructura viva con inserciones y búsquedas continuas en $O(\log n)$.

Anexos

Video educativo que explica visualmente los algoritmos de ordenamiento

<https://www.youtube.com/watch?v=ZZuD6iUe3Pc>

Repositorio de GitHub con el código fuente completo:

<https://github.com/Camest24/PracticaSorting>